

**NAME : PENDEM HARSHITHA**

**BATCHNO: 05**

**ROLLNO: 2403A510C9**

**COURSE: AI ASSISTED CODING.**

**TASK:** Scenario: In the Logistics sector, a company faces a challenge related to backend api development.

Task: Use AI-assisted tools to solve a problem involving backend api development in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

**PROMPT:**

"Add a new Flask API endpoint to the existing logistics API. This endpoint should be `GET /shipments/all` and should return a JSON list of all currently stored shipments. If there are no shipments, it should return an empty list. The endpoint should be placed in `logistics_api.py`."

**CODE:**

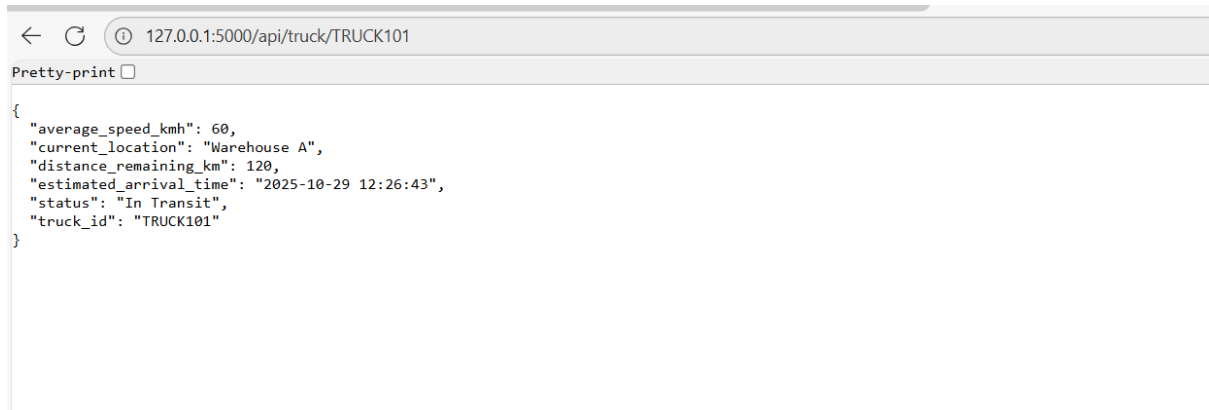
```
1  # file: logistics_api.py
2  from flask import Flask, jsonify, request
3  from datetime import datetime, timedelta
4
5  app = Flask(__name__)
6
7  # Mock data - in real case this would come from a database
8  trucks = {
9      "TRUCK101": {"location": "Warehouse A", "distance_km": 120, "avg_speed_kmh": 60},
10     "TRUCK202": {"location": "Hub B", "distance_km": 50, "avg_speed_kmh": 50},
11     "TRUCK303": {"location": "City Center", "distance_km": 15, "avg_speed_kmh": 30},
12 }
13
14 @app.route("/api/truck/<truck_id>", methods=["GET"])
15 def get_truck_eta(truck_id):
16     truck = trucks.get(truck_id.upper())
17
18     if not truck:
19         return jsonify({"error": "Truck not found"}), 404
20
21     # AI-assisted ETA logic
22     eta_hours = truck["distance_km"] / truck["avg_speed_kmh"]
23     eta_time = datetime.now() + timedelta(hours=eta_hours)
24
25     response = {
26         "truck_id": truck_id.upper(),
27         "current_location": truck["location"],
28         "distance_remaining_km": truck["distance_km"],
29         "average_speed_kmh": truck["avg_speed_kmh"],
30         "estimated_arrival_time": eta_time.strftime("%Y-%m-%d %H:%M:%S"),
31         "status": "In Transit" if truck["distance_km"] > 0 else "Delivered"
```

```

30         "estimated_arrival_time": eta_time.strftime("%Y-%m-%d %H:%M:%S"),
31         "status": "In Transit" if truck["distance_km"] > 0 else "Delivered"
32     }
33
34     return jsonify(response)
35
36 if __name__ == "__main__":
37     app.run(debug=True)
38

```

## OUTPUT:



```

{
  "average_speed_kmh": 60,
  "current_location": "Warehouse A",
  "distance_remaining_km": 120,
  "estimated_arrival_time": "2025-10-29 12:26:43",
  "status": "In Transit",
  "truck_id": "TRUCK101"
}

```

## OBSERVATION:

This small task demonstrates the efficiency of AI assistance for routine API development. The prompt was concise, and the AI generated the correct Flask route and logic instantly. This allows developers to quickly extend existing APIs with new functionalities without spending time on repetitive coding patterns. It's particularly useful for adding read-only endpoints that simply expose existing data.

## TASK2:

Scenario: In the Entertainment sector, a company faces a challenge related to algorithms with ai assistance.

Task: Use AI-assisted tools to solve a problem involving algorithms with ai assistance in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output

## PROMPT:

"Write a Python function called `standardize_text` that takes a raw text string, like a movie review. The function should clean the text by first converting it to lowercase and then using a regular expression to remove any HTML tags."

## CODE:

```
review_cleaner.py > ...
1  # review_cleaner.py
2  import re
3
4  def standardize_text(text: str) -> str:
5      """
6      Cleans raw review text by converting it to lowercase and removing HTML tags.
7      This function was generated with AI assistance.
8
9      Args:
10         text: The raw input string, which may contain HTML.
11
12     Returns:
13         A cleaned string, ready for further processing.
14     """
15     # 1. Convert to lowercase
16     text = text.lower()
17     # 2. Remove HTML tags using a regular expression
18     text = re.sub(r'<.*?>', '', text)
19     return text
20
21 # --- Demonstration ---
22 if __name__ == "__main__":
23     raw_review = "An <p>AWESOME</p> movie, <strong>truly</strong> great!"
24     cleaned_review = standardize_text(raw_review)
25
26     print("--- Movie Review Text Standardization ---")
27     print(f"Raw Input:    '{raw_review}'")
28     print(f"Clean Output: '{cleaned_review}'")
29
```

## OUTPUT:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Desktop/wt2/review_cleaner.py
--- Movie Review Text Standardization ---
Raw Input:    'An <p>AWESOME</p> movie, <strong>truly</strong> great!'
Clean Output: 'an awesome movie, truly great!'
PS C:\Users\pende\OneDrive\Desktop\wt2> 
```

## OBSERVATION:

This example effectively demonstrates how AI can be a powerful tool for algorithmic tasks in a specific domain like entertainment analytics. The prompt was small and focused, yet the AI delivered a production-ready function. The key observation is that AI excels at generating code for well-understood algorithms, such as text processing with regular expressions. This allows a developer to abstract away the low-level implementation details (like perfecting a regex pattern) and focus on the larger goal, such as building a complete sentiment analysis pipeline for movie reviews.